



École Polytechnique Fédérale de Lausanne

Tile-Based Decompression in Compiled GPU Engines

by Ananya Gupta

Semester Project Report

Approved by the Examining Committee:

Prof. Dr. Anastasia Ailamaki
Thesis Advisor

Dr. Antonio Boffa
Thesis Supervisor

EPFL IC IINFCOM DIAS
BC 222 (Bâtiment BC)
Station 14
CH-1015 Lausanne

June 12, 2026

Acknowledgments

I would like to thank my supervisor, Dr. Antonio Boffa, for his patient guidance throughout the semester and for the opportunity to contribute to a research paper alongside the project.

I am also deeply grateful to Prof. Anastasia Ailamaki for welcoming me into the lab and giving me the chance to take part in the research discussions and weekly meetings that shaped how I think about database systems.

Finally, my thanks to all the members of the lab for the everyday conversations, feedback, and discussions that made this semester a genuinely enjoyable learning experience.

Lausanne, June 12, 2026

Ananya Gupta

Abstract

Modern GPU analytical engines achieve record-setting performance when the working set fits in High-Bandwidth Memory, but HBM capacity is fixed and its per-dollar cost has stagnated across four generations. Keeping data compressed in GPU memory addresses both constraints, yet the two existing responses are unsatisfying: engines that decompress eagerly into HBM waste the bandwidth they meant to save, and engines that execute directly on compressed data hard-wire decompression into a single execution model, so each codec is rewritten per system. The intersection of *compiled, tuple-at-a-time* GPU execution with *modular, plug-and-play* compression backends has remained unexplored.

This project (along with the companion paper [2]) attempts to close that gap by introducing a Modular Decompression Operator into Proteus, a JIT-compiled tuple-at-a-time GPU query engine. The operator commits to a narrow two-hook backend contract (`resolve_metadata`, `decompress_tile`) and exposes on-chip placement and tile/thread shape as orthogonal configuration knobs. My empirical contribution has two complementary components: an **end-to-end analysis of the JIT-emitted operator inside Proteus across SSB Q1.1–Q4.1**, and a **standalone CUDA micro-benchmark** that realises the same contract by hand on a hard-coded SSB Q1.1 functor and acts as a hardware-level oracle for cross-comparison with the JIT path at matched configuration.

The Proteus analysis shows that fused on-chip decompression opens a previously empty region of the GPU memory-footprint–query-time Pareto frontier across all four queries, and reveals what actually governs the cost of compressed query execution on GPUs - which on-chip resources matter, and how the workload and the query shape their pressure on those resources. The micro-benchmark comparison on Q1.1 additionally exposes where the JIT-emitted operator still falls behind a hand-written equivalent built against the same contract.

Contents

Acknowledgments	2
Abstract	3
1 Introduction	5
2 Background	9
2.1 GPU architecture	9
2.2 GPU analytical query engines	11
2.3 Compiled tuple-at-a-time execution in Proteus	12
2.4 Lightweight columnar compression: FLBP and RFOR	13
2.5 Profiling vocabulary	14
3 Design and Implementation	16
3.1 The Modular Decompression Operator inside Proteus	16
3.2 Standalone benchmark	19
3.2.1 Architecture	19
3.2.2 Codec backends	22
3.2.3 Metadata management	23
3.3 Profiling pipeline	24
4 Evaluation	25
4.1 Setup	25
4.2 End-to-end footprint vs. query time	26
4.3 Placement and launch-configuration sensitivity	27
4.4 Projectivity sensitivity	30
4.5 Cross-validation: the standalone benchmark as oracle	33
4.6 Mechanism: the eligible-warp gap	34
4.7 Synthesis and limitations	35
5 Conclusion	36
Bibliography	38

Chapter 1

Introduction

GPU analytical engines and the HBM constraint. Modern GPUs combine massive parallelism with multi-terabyte-per-second memory bandwidth, and recent GPU-native SQL engines such as Proteus [11], HeavyDB [6], and Sirius [17] translate these hardware advantages into record-setting end-to-end analytical performance. This progress builds on more than a decade of GPU database research showing that large speedups for relational operators are achievable when those operators are mapped carefully to GPU execution and memory behaviour [5, 14, 15]. Achieving such performance, however, still depends on one fragile condition: the working set must fit within the on-device High-Bandwidth Memory (HBM) budget.

The HBM affordability gap. This condition is increasingly fragile. HBM is a fixed device resource: unlike CPU DRAM, it cannot be expanded without changing the GPU class. Cloud pricing reflects this scarcity directly. As shown in Figure 1.1, across the last four NVIDIA generations GPU device memory has grown by 3.6 $\times$ (from 80 GB on the A100 to 288 GB on the B300), yet the number of GB-hours of memory a dollar buys - averaged across four hyperscalers and one marketplace aggregator - sits below the A100 baseline. Capacity scales; affordability does not. The same constraint reappears inside the data center, where mechanisms such as Multi-Instance GPU [8] and Multi-Process Service [9] partition a single GPU and shrink the effective budget per workload. Efficient use of HBM is therefore not an optimisation: it is a precondition for cost-effective GPU analytics.

Existing approaches and their limits. Two families of designs respond to this pressure. The first decompresses columnar data eagerly into HBM and runs uncompressed operators downstream; this is the model adopted by Proteus' default compressed path [11] and by competing GPU engines that rely on nvCOMP and similar libraries [12]. The second executes operators directly on compressed representations: Crystal [15] and its compressed extensions [16] pioneered the tile-at-a-time model in which a thread block cooperatively decodes a tile into shared memory, and recent work [1, 7]

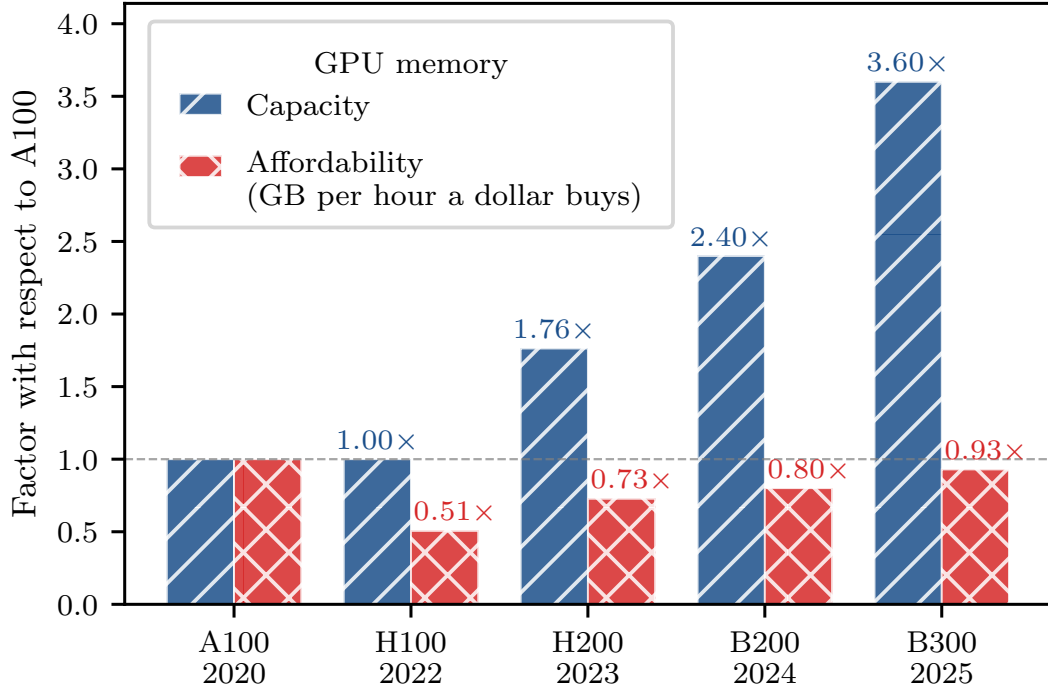


Figure 1.1: Affordability gap between GPU memory capacity and the memory each dollar buys. From A100 to B300, device memory grows by 3.6 $\times$, while the number of GB-hours of memory a dollar buys (averaged across four hyperscalers and one marketplace aggregator) sits below A100 levels. Capacity scales; affordability does not.

extends the idea to tensor runtimes and SIMD-friendly encodings such as FastLanes. The first family wastes HBM bandwidth by materialising inflated values; the second hard-wires decompression into a specific execution model (block-cooperative CUDA, or tensor primitives), so each new codec requires per-codec implementation effort and cannot reuse the optimisations of a compiled query engine. The intersection of *compiled, tuple-at-a-time* execution with *modular, plug-and-play compression backends* has remained unexplored.

A modular operator inside a compiled engine. The companion paper [2] and this project closes that gap by introducing a Modular Decompression Operator (MDO) into Proteus, a JIT-compiled tuple-at-a-time GPU query engine. The operator commits to a narrow two-hook contract: a per-tile `resolve_metadata()` hook that interprets the tile header, and a cooperative `decompress_tile()` hook that decodes one tile’s worth of values. Everything compression-specific lives behind those two hooks; downstream relational operators consume ordinary tuple bindings and remain oblivious to whether the data came from a raw column, RFOR, or FastLanes bitpack (FLBP). My role in this project is to implement the two backends as hand-written CUDA kernels in a standalone

benchmark, to characterise the JIT-emitted Proteus operator across SSB Q1.1–Q4.1, to cross-validate it on Q1.1 against the hand-written equivalent, and to study empirically how on-chip placement (staging decoded values in shared memory versus in registers) and tile/thread configuration affect end-to-end query cost.

A controlled testbed for the operator’s hardware-level behaviour. The MDO inside Proteus is JIT-emitted, so any kernel-level measurement captures the codec, the operator contract, the LLVM IR emission shape, and the surrounding pipeline simultaneously. To separate these effects, I built a standalone CUDA micro-benchmark, `standalone_ssb`, that implements the same two-hook contract as a compile-time C++20 concept (`ModeConfig`) and runs the same SSB Q1.1 Query plan against the same data. The benchmark serves three purposes throughout the report: (i) it is the executable embodiment of the operator contract, proving the boundary is implementable independent of JIT; (ii) it is a hardware-level oracle against which Proteus numbers are compared, so any gap between the two can be identified and investigated; and (iii) it is a fast configuration scanner, recompiling in seconds, that makes the design-space sweep tractable before optimizing into Proteus.

Project statement. Lightweight decompression can be a first-class, JIT-fused, plug-and-play operator of a compiled tuple-at-a-time GPU query engine, provided that the on-chip placement of decoded values and the tile/thread configuration are chosen with respect to the observed trade-offs among register pressure, shared-memory occupancy, and global-memory traffic.

Result highlights. Two findings frame the evaluation. First, on most of the SSB queries we study, our approach sits closer to the memory-footprint–query-time Pareto frontier than every prior alternative, with further gains available by tuning the configuration to the query at hand: the best setting depends on the compression-method, the number of columns the query reads, and the query’s overall complexity, so a one-size-fits-all configuration is not the right target. Second, at the same configuration and codec, the JIT-fused operator inside Proteus runs slightly slower than the equivalent hand-written kernel - not because of the algorithm, but because Proteus’ code generator emits the per-column decoding logic one column at a time, which prevents the GPU from overlapping memory accesses across columns. We identify this as the next unexploited optimisation opportunity inside the operator.

Contributions. This report makes two concrete contributions:

1. **Hand-written CUDA implementations of the two codec backends**, RFOR and FLBP, for both placement strategies (shared-memory and register-resident decode). The implementations live inside a standalone benchmark that compresses each column on the CPU, uploads it to the GPU, decompresses on-device, and runs a fixed SSB Q1.1 query plan over the decoded

values, with round-trip correctness checks guarding each codec. The benchmark functions both as a hardware-level oracle against Proteus' JIT-emitted counterpart and as a reusable testbed for future codecs.

2. **An end-to-end empirical study** of the JIT-emitted operator inside Proteus across SSB Q1.1–Q4.1, supported by a set of profiling and analysis scripts built on top of NVIDIA Nsight Compute and `ptxas`, that drives the experimental analysis section of the companion paper [2]. The study reveals what actually governs the cost of compressed query execution on GPUs - which on-chip resources matter, how the workload and the query shape their pressure on those resources, and where the operator's configuration can be tuned more deliberately once the query is known.

Chapter 2

Background

This chapter introduces the pieces of context a reader needs to follow the rest of the report: the GPU hardware (Section 2.1), GPU analytical query engines as a category (Section 2.2), Proteus' execution model as a specific instance (Section 2.3), the two compression schemes whose decoders we plug into it (Section 2.4), and the small set of profiling metrics that make our experimental claims falsifiable (Section 2.5).

2.1 GPU architecture

Compute hierarchy. A modern NVIDIA GPU (Figure 2.1) is organised as a small number of *Streaming Multiprocessors* (SMs); the A40 used throughout this report has 84, each containing four warp schedulers and 65,536 registers. Programs are launched in a grid of *thread blocks* (CTAs), and each block contains many *threads*. The hardware groups threads in lock-step bundles of 32 called *warps*; a warp is the schedulable unit of work, and the four schedulers on each SM each pick one ready warp per cycle to issue an instruction. Threads within a warp share an instruction stream; threads in different warps do not. A block is dispatched in its entirety to a single SM and stays resident until all its threads exit.

Memory hierarchy. Off-chip, the GPU has *High-Bandwidth Memory* (HBM): the A40 ships with 48 GB at ≈ 696 GB/s. From the CUDA programming model HBM is exposed as *global memory*, and the rest of this report uses the two terms interchangeably. HBM is the only place where inter-block state can persist; writing decoded tiles back to HBM is the eager-decompression baseline this report contrasts with. *Shared memory* is a per-SM scratchpad that all threads in a block can access at near-register speed - on sm_86 up to 100 KB per SM, of which a single block can use at most 99 KB; it is the natural staging area for cooperative decoders. *Registers* are per-thread and offer single-cycle

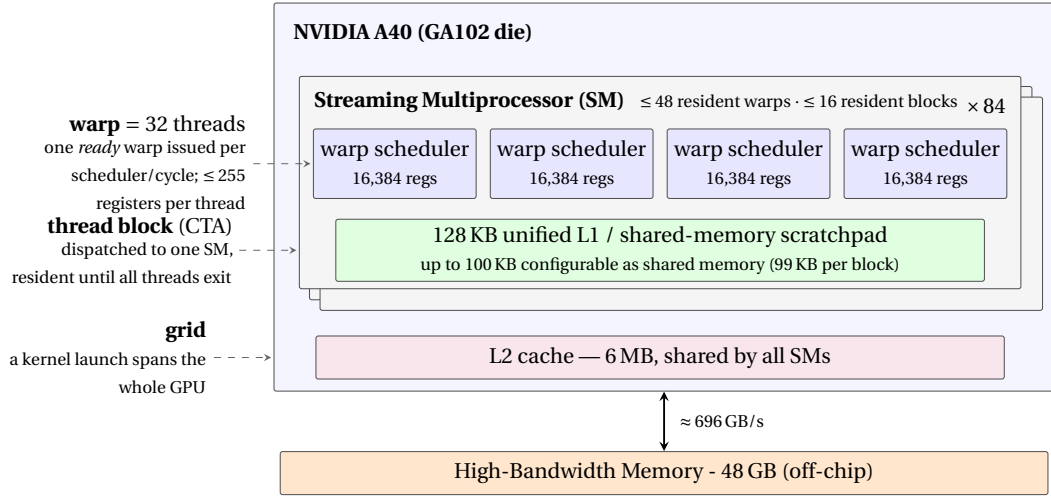


Figure 2.1: Compute and memory hierarchy of the NVIDIA A40 used throughout this report, annotated (dashed) with the software unit that maps onto each hardware level. Each of the 84 SMs hosts four warp schedulers with a 16,384-register file slice each, plus a 128 KB L1/shared-memory scratchpad of which up to 100 KB is configurable as shared memory; an SM hosts at most 48 resident warps and 16 resident blocks. All SMs share a 6 MB L2 and 48 GB of HBM at ≈ 696 GB/s. Redrawn from the GA102 whitepaper [13] and the CUDA C++ Programming Guide [4].

access, but their count per thread is bounded (255 on sm_86), and the per-thread budget is what limits the number of warps that can fit on an SM.

Occupancy and its step function. The number of warps a scheduler can dispatch from is bounded by how many blocks the SM can host concurrently. The hardware admits an *integer* number of blocks per SM, chosen as the min over four limits:

- the per-SM **thread budget**: $\lfloor \max_threads_per_SM / BT \rfloor$ (48 warps = 1,536 threads on sm_86);
- the per-SM **shared-memory budget**: $\lfloor \max_smem_per_SM / smem_per_block \rfloor$ (≈ 100 KB on sm_86);
- the per-SM **register-file budget**: $\lfloor \max_regs_per_SM / (regs_per_thread \times BT) \rfloor$ (65,536 registers on sm_86);
- the hard **resident-block cap** (16 blocks per SM on sm_86; Figure 2.1).

Figure 2.2 visualises the four-way min on a representative configuration.

Of these four limits, the thread budget and the resident-block cap are fixed by the launch shape alone; the shared-memory and register-file budgets are the only ones that vary with the codec,

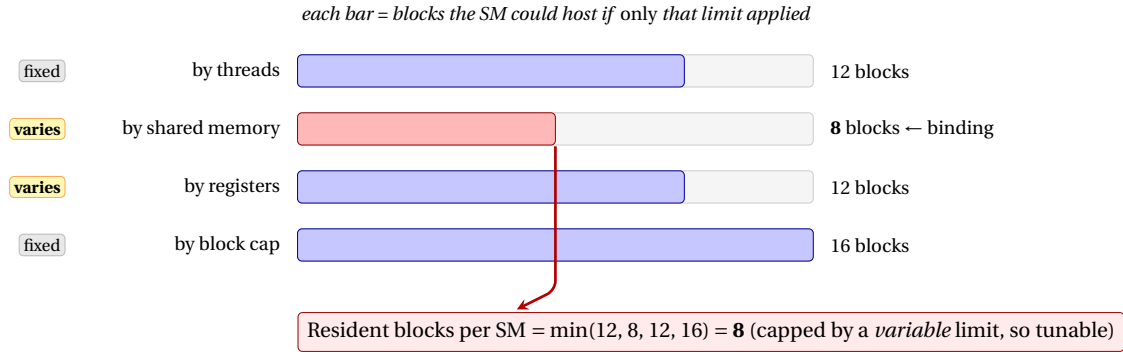


Figure 2.2: Each bar = blocks the SM could host under a single limit; the hardware admits the min. Yellow badges mark the two tunable limits (shared memory, registers); gray badges mark the fixed ones (thread budget, block cap). Illustrative numbers: shared memory binds at 8 blocks while the other three would each accommodate ≥ 12 .

placement, and projectivity choices the rest of this report sweeps. Maximising occupancy therefore reduces to balancing those two against the launch geometry. The consequence used throughout the evaluation is that occupancy is not continuous in either resource: crossing the threshold of any one budget by a single byte or single register can drop the achieved occupancy by an entire resident block. This is why a small change in tile size or projected-column count can move a configuration from a winning to a losing point without any change in algorithm.

2.2 GPU analytical query engines

The category. GPU analytical query engines run relational queries on the GPU’s parallel compute and HBM bandwidth, rather than relying on the CPU’s larger caches and richer scalar pipelines. Their appeal is empirical: across recent NVIDIA generations, an A40 sustains ≈ 696 GB/s of HBM bandwidth versus the ≈ 200 GB/s of its host CPU’s main memory, and the GPU’s relational kernels can translate that bandwidth into shorter end-to-end query times on workloads where the working set fits in HBM [14, 15].

Two execution-model families. GPU engines partition along the same vectorised-versus-compiled axis that organises CPU engines. *Vectorised* or *tile-based* engines (Crystal [15, 16], the tensor-runtime line of TQP [7], Sirius [17]) launch one kernel per operator over a tile of rows or a tensor of values, keeping cross-operator state on chip and exchanging it through shared memory. *Compiled, tuple-at-a-time* engines (Proteus [3, 11], DogQC [5], HeavyDB [6]) generate a single fused kernel per pipeline that pushes individual tuples through a chain of operator code.

Where this report sits. Proteus is the specific instance this report works in: a compiled tuple-at-a-time GPU engine in the same lineage as HyPer’s CPU-side compiler work [10], but with hardware-conscious operators that match the GPU’s SIMT shape. The Modular Decompression Operator described in Chapter 3 extends Proteus’ operator vocabulary without forcing it into either of the alternative engine families.

2.3 Compiled tuple-at-a-time execution in Proteus

Compiled query engines. A *compiled* query engine translates each query plan into native machine code, rather than interpreting it through a tree of operators. The technique was popularised on the CPU side by HyPer [10]: each operator implements two methods, `produce()` and `consume()`, which together emit a single loop nest in which tuples flow through the entire pipeline without ever being materialised between operators. The compiled code keeps intermediate values in registers and, crucially, never has to dispatch on operator type at run time.

Proteus. Proteus [3, 11] is a JIT-compiled GPU query engine in the same lineage. For each query it walks the plan, emits LLVM IR for the pipeline, and compiles that IR down to one CUDA kernel per *pipeline* (the longest sequence of operators with no pipeline break). Within a pipeline every thread executes the same compiled code on a different input tuple. This is what we mean by *tuple-at-a-time* GPU execution: parallelism is across tuples, not across operators, and the body of the kernel is a fused chain of operator code rather than a sequence of kernel launches [5].

Block-at-a-time data movement. Although execution is tuple-at-a-time, the unit of data movement and storage is coarser: Proteus partitions each column into fixed-size 2 MiB *blocks*, which are the granularity at which the engine overlaps host-to-GPU transfer with computation [11]. A *Block-to-Tuples* (BtT) operator sits at the boundary between storage and pipeline: it consumes a block of raw values and emits a stream of tuple bindings that the rest of the compiled pipeline can process. The BtT operator is the single point in the plan where storage and execution meet, which makes it the natural place to inject decompression: a *CompressedBlockToTuples* (CB2T) operator extends BtT by emitting per-tile decode IR before producing the tuple bindings.

Why fused decompression is mandatory, not optional. Because Proteus compiles each pipeline into a single kernel, there is no opportunity to run a stand-alone "decompression kernel" between scan and filter: the JIT either emits decompression IR inside the same kernel, or it materialises decompressed values to HBM and reads them back, which is the eager-decompression baseline this project aims to avoid. The modular-decompression operator introduced in Chapter 3 is therefore

not just a software-engineering refactor; it is the structural prerequisite for keeping decode inside the fused kernel.

2.4 Lightweight columnar compression: FLBP and RFOR

Lightweight, tile-decodable encodings. A compression scheme is *lightweight* in the sense used by recent GPU work [1, 16] when (i) a fixed-size tile of encoded data can be decoded independently of any other tile, (ii) the per-tile metadata is small and uniform, and (iii) the decode step is data-parallel across the threads of one block. These three properties let a single thread block decode one tile cooperatively, with no cross-block synchronisation and no global decompressed buffer. All encodings considered in this report share these properties and rest on two atoms common across the lightweight family: Frame-of-Reference (*FOR*), which stores each tile’s minimum value once and represents the remainder as residuals; and *bit-packing*, which stores residuals using only the bits required by the tile’s maximum residual width. The two schemes we focus on combine these atoms differently.

FastLanes Bitpack (FLBP). FLBP [1] fixes the tile size at 1024 values and lays them out as 32 *lanes* of 32 positions each. For tile bitwidth bw , the packed payload is $bw \times 32$ uint32 words, transposed so that word $k \cdot 32 + l$ holds the bits of position k of lane l . The resulting decode is purely arithmetic: thread t in a warp computes its lane as $t \bmod 32$, its position from a bit offset, and reads exactly one packed word for any single decoded value. Per-tile metadata is three fields (FOR base, bitwidth, element count), and a single warp suffices to materialise the whole tile with no inter-thread communication. FLBP is the design point optimised for raw decode speed at the cost of compression ratio: it does not exploit run structure or cross-element redundancy.

RFOR (RLE + FOR + bitpack). RFOR [16] compresses two parallel streams per tile: a *value* stream containing unique values and a *run-length* stream encoding how many times each value repeats. Each stream is independently FOR-encoded and bit-packed. A tile begins with a 24-byte header carrying six fields (FOR base, bitwidth, element count for the value stream, and the same triple for the run-length stream), followed by the two bit-packed payloads back-to-back. Decoding a tile is a multi-step cooperative process: the block stages the value stream into shared memory, bit-unpacks and adds the FOR base, then repeats the same sequence on the run-length stream, and finally runs a block-wide prefix scan to expand runs back to per-element values. The richer representation lets RFOR achieve higher compression ratios on the columns [11] but raises both metadata cost and per-tile shared-memory pressure.

Why two codecs. FLBP and RFOR occupy opposite corners of the compression-ratio/decode-speed plane and exercise different parts of the operator boundary: FLBP stresses register pressure (its decode is mostly arithmetic on per-thread values), RFOR stresses shared-memory pressure (it stages two streams plus scratch buffers per tile). Together they form the smallest pair that surfaces every trade-off the rest of the report studies.

2.5 Profiling vocabulary

Nsight Compute (ncu). Throughout Chapter 4 we report numbers from NVIDIA Nsight Compute (ncu), a kernel-level profiler that replays each kernel under hardware counters. Three sections matter for this work: OCCUPANCY (resident vs. maximum warps), SCHEDULERSTATS (per-cycle warp issue behaviour), and MEMORYWORKLOADANALYSIS (DRAM read/write throughput, L1/L2 hit rates).

Achieved occupancy vs. eligible warps. *Achieved occupancy* is the time-averaged ratio of active warps to the hardware maximum. It is a necessary but not sufficient condition for throughput: a warp can be resident but stalled on a memory dependency, contributing to occupancy without contributing to instruction issue. The metric that captures the latter is *eligible warps per scheduler*: the average number of warps ready to issue an instruction at any cycle, measured per warp scheduler (an SM has four on sm_86). If this number is below one the scheduler is regularly idle even when occupancy is high. Section 4.6 reports a striking instance: at matched configuration, the JIT-fused pipeline records 0.56 eligible warps per scheduler while the hand-written standalone kernel records 1.31, and the gap explains a large fraction of the runtime difference.

PTX assembler (ptxas) reports. The NVIDIA PTX assembler emits per-kernel reports that ncu does not expose directly. Two fields are used in the evaluation: the per-thread register count (which feeds the occupancy step function described above) and the spill stores/loads count (which is non-zero when ptxas cannot keep all live values in registers and falls back to local memory). A custom pipeline in `results/scripts/` parses these reports and joins them with the ncu numbers.

LPT critical-path estimation. Aggregate kernel time hides imbalance across blocks. The standalone benchmark therefore records, per block, the time spent in each of the three pipeline phases (decompress, query, reduce) using the device-side `globaltimer64()` intrinsic, then simulates an SM-level scheduler using the Longest-Processing-Time (LPT) heuristic to estimate the critical-path latency given the number of resident blocks. The result is a first-order proxy for wall time that separates algorithmic cost from launch-geometry effects.

How this chapter feeds the rest. Section 2.1 establishes the GPU hardware vocabulary - SMs, warps, registers, shared memory, HBM, and the occupancy step function - on which every later resource-pressure argument depends. Section 2.2 places Proteus within the broader category of GPU analytical engines, motivating the choice to extend its JIT-compiled tuple-at-a-time model rather than build a new one. Section 2.3 fixes Proteus' execution context specifically and motivates the design in Chapter 3. Section 2.4 introduces the two codecs whose backends Section 3.2.2 walks through. Finally, Section 2.5 gives the metrics needed to read the mechanism-level findings in Section 4.6 as anything more than coincidences.

Chapter 3

Design and Implementation

Chapter 2 explained *what* a compiled tuple-at-a-time GPU engine looks like, what FLBP and RFOR encode, and what the GPU memory hierarchy offers. This chapter is about the design under which those raw materials are exercised in the rest of the report. It has two parts: Section 3.1 sketches the Modular Decompression Operator (MDO) that the companion paper [2] introduces inside Proteus - just enough context to read Chapter 4; Section 3.2 describes the standalone implementation I built against the same contract, and the role it plays as a controlled-experiment counterpart to the JIT-emitted operator.

3.1 The Modular Decompression Operator inside Proteus

I did not design or implement the MDO; it is the contribution of the companion paper [2], and the full design lives there. This section condenses the parts of that design that the rest of this report depends on: the composability goal, the two-hook backend contract, and the two execution knobs the operator exposes to the plan.

Composability for decompression. The MDO commits to two sub-properties that together define what composability means inside a compiled GPU engine. **(i) Interface substitutability:** a compressed column presents the same logical attribute interface as an uncompressed one; downstream relational operators consume values through the engine’s standard scan/tuple interface, not through compression-algorithm-specific APIs. **(ii) Decompression encapsulation:** compressed representation details - metadata layout, bit packing, run structure, tile decoding - are confined to a reusable backend layer that no relational operator code ever has to touch.

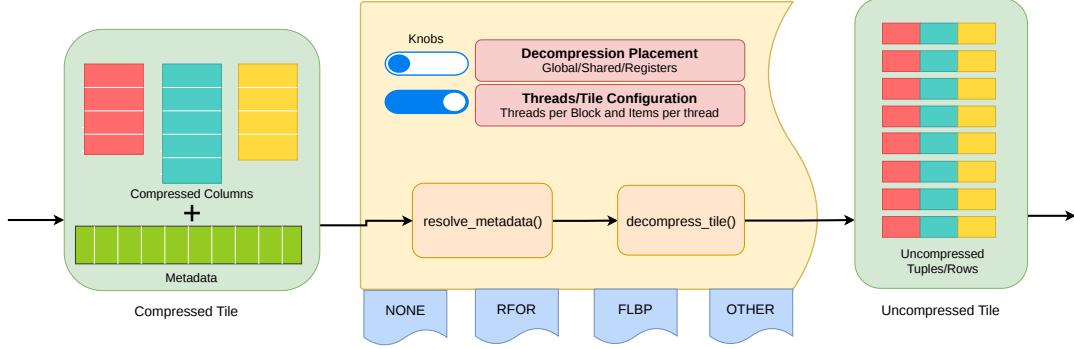


Figure 3.1: The Modular Decompression Operator sits inside a fused query kernel. Compressed blocks enter from the left; the operator pulls a backend from a pool of codecs (bottom-left), each contributing only `resolve_metadata()` and `decompress_tile()`; the right-hand interface is the engine’s standard tuple stream, identical to the uncompressed path. Execution knobs in the bottom-right control where decoded values live and the tile/thread shape. The full design is in the companion paper [2].

The two-hook backend contract. Substitutability and encapsulation are realised by a narrow backend contract with exactly two hooks (Figure 3.1):

- `resolve_metadata(tile)` - runs on a single block leader (one thread per CTA), parses the per-tile header, and broadcasts the resolved fields (FOR base, bitwidth, element count, pointers to the packed payloads) into shared memory.
- `decompress_tile(tile)` - executes cooperatively across all threads of the block, reads the metadata, and emits one decoded tile per column either into shared memory or directly into per-thread registers.

The asymmetric cooperation model is the natural shape of lightweight GPU decompression: metadata is small and broadcast-friendly; decoding is wide and data-parallel. Confining each codec to these two hooks means a new codec is added by writing only its format-specific decoder; everything downstream of the operator boundary remains codec-oblivious.

Placement as a configuration knob. The MDO exposes the destination of decoded values as a per-plan knob rather than baking it in. Three placement modes, named after where the decoded values live, map directly onto the three on-chip address spaces of Section 2.1:

- `pre_decom` (pre-decompression into global memory): decoded tiles are written back to HBM and re-read by downstream operators. This is the *eager-decompression baseline* this report contrasts the two fused placements against, and is the default behaviour of engines that decompress before query execution.

- `decom_op` (decode-into-the-operator’s shared-memory buffer): decoded values land in a shared-memory staging area persistent across the rest of the pipeline body, so downstream operators read from shared memory and the JIT emits ordinary load instructions against shared addresses.
- `decom_reg` (decode-into-registers): decoded values are written straight into per-thread registers consumed inside the same kernel iteration, with no intermediate staging.

Table 3.1 summarises the three modes and names the corresponding standalone-benchmark realisation that Section 3.2.2 walks through.

Table 3.1: The three placement modes the MDO exposes, mapped to the on-chip address space they target and the corresponding standalone benchmark mode (Section 3.2.2). The `pre_decom` mode is the eager-decompression baseline; the two fused modes (`decom_op`, `decom_reg`) are the design space this report’s evaluation actually navigates.

Placement flag	Address space	Standalone bench mode	Fused?
<code>pre_decom</code>	global memory (HBM)	RFOR_GLOBAL, FLBP_GLOBAL	no
<code>decom_op</code>	shared memory	RFOR_SHARED	yes
<code>decom_reg</code>	per-thread registers	FLBP_REG	yes

The `decom_op` and `decom_reg` modes are both emitted by the same `CompressedBlockToTuples` JIT-fused path; the choice between them is a flag, not a code branch in the engine [11]. The `pre_decom` baseline follows a different implementation path: a separate pre-decompression step writes decoded tiles back to HBM, and a downstream uncompressed query kernel consumes them. The interesting trade-off, and the one this report studies, is between the two fused placements; `pre_decom` serves as a footprint and bandwidth reference point against which the fused modes are evaluated. Within the fused pair the trade-off is sharp and not codec-local: shared-memory placement keeps register pressure low but consumes scratchpad budget that grows linearly with the number of projected columns N and the tile size T (each decoded `int32` is 4 bytes, so the staging area is $4 \cdot T \cdot N$ bytes per tile); register placement avoids that cost but pushes ptxas towards spilling, which silently moves values through global memory and undoes the savings. The right fused placement depends on the codec, the number of projected columns, the tile size, and the downstream operator graph, so no single placement wins across the plan. This is the design rationale that the projectivity sweep in Section 4.4 exercises empirically.

Tile and thread-block shape as compile-time parameters. The kernel shape is exposed as a triple (BT, IPT, T) : BT block-threads per CTA, IPT items per thread, T tile size. For FLBP the product $BT \cdot IPT = 1024$ is fixed by the FastLanes vector size, so only T and the (BT, IPT) partition vary; RFOR has no such invariant. CUDA forces these to be compile-time (register-array declarations, CUB type instantiation, static shared-memory size, `#pragma unroll` bounds), so each (BT, IPT)

pair compiles to a distinct binary - a fact that matters for sweeping the configuration space efficiently, and that motivates the standalone implementation in Section 3.2 below.

3.2 Standalone benchmark

Why a second implementation. The MDO inside Proteus mixes four effects in any single kernel-level measurement: the codec, the operator contract, the LLVM IR emission shape, and the downstream relational operators (filter, projection, hash probe, aggregation) that consume the decoded values inside the same fused kernel. To attribute observed behaviour to one of these factors, we needed a controlled experiment in which the codec and the contract are the same but the JIT and the downstream pipeline are absent. The standalone benchmark `standalone_ssb` is exactly that: a hand-written CUDA executable that implements the same two-hook contract by other means, runs on the same SSB Q1.1 data, and reports per-block phase timings. Without this second track, the eligible-warp gap reported in Section 4.6 would have been ambiguous between codec, contract, and JIT-emission cost; with it, the attribution is clean.

Four roles in the rest of the report. The standalone benchmark plays four distinct roles, each invoked in a different chapter. First, it is the *executable embodiment of the operator contract*: its existence demonstrates that the two-hook boundary is implementable independently of JIT. Second, it is a *hardware-level oracle*: at matched configuration, codec, and data, any gap between standalone and Proteus numbers is attributable to JIT and pipeline context, used throughout Chapter 4. Third, it is a *configuration-space scanner* for the matched-Q1.1 setup used in Section 4.5: each (*BT*, *IPT*, *T*) point recompiles in seconds, so the design-space sweep happens here first and the winning configuration is what the Proteus cross-comparison runs against. Fourth, it is a *differential-diagnosis tool*: when Proteus underperforms, the benchmark separates kernel-source problems from JIT-emission problems, used in Section 4.6.

3.2.1 Architecture

Directory layout. The benchmark is split into six top-level concerns (Figure 3.2). Each compression strategy is a single self-contained `mode/name.cuh` file. Host-side encoders live in `compress/`, device-side decoders in `decompressor/`. The execution-agnostic harness is in `runner/`, shared scalar types in `common/`, and GTest round-trip tests in `tests/`. Adding a new codec means writing a new triple of files plus a one-line entry in the enum.

The contract realised as a C++20 concept. Each codec backend in the benchmark is a struct that satisfies a single C++20 concept, `ModeConfig`. The concept members map directly onto the

```

apps/standalones/adm/adm-encode-bench/modular/
main.cpp           ExecMode parsing + per-mode dispatch
CMakeLists.txt     standalone_ssb + tests targets
run.sh, sweep.sh   one-shot / sweep entry points
profile_modular.sh ncu profiling driver
common/
  types.cuh        sum_type, nsecs, globaltimer64()
compress/
  encoding_concept.cuh ModeConfig C++20 concept
  fl_bitpack_compress.hpp
  fl_delta_compress.hpp
  rfor_compress.hpp
decompressor/
  fl_bitpack_decompressor.cuh
  fl_delta_decompressor.cuh
  rle_binpack_decompressor.cuh
mode/
  fl_bitpack.cuh      FLBP kernel + ModeConfig
  fl_delta.cuh
  rfor_global.cuh     RFOR with global writeback
  rfor_shared.cuh     RFOR with shared-memory placement
  uncompressed_row.cuh baseline
runner/
  kernel_runner.cuh   run_mode<C>(): N-iteration timer + LPT
  run_config.cuh      ExecMode enum
  metrics.cuh         per-block phase timing aggregation
  correctness.cuh     sum check against expected revenue
tests/
  test_decompressors.cu GTest round-trip

```

Figure 3.2: Layout of the standalone benchmark. The split between `mode/` (kernel + `ModeConfig`) and `decompressor/` (device-side decoder) is what lets a single decoder be reused across multiple placement modes (e.g. the FLBP decoder is the same in `decom_op` and `decom_reg` variants).

operator-level decisions of Section 3.1, as shown in Table 3.2. The benchmark cannot deviate from the contract because the concept fails to type-check otherwise, so the mapping is enforced by the compiler rather than by convention - which is what makes the standalone numbers comparable to Proteus’ on the same axes.

The harness. The benchmark harness is a single function template, `run_mode<C>()` in `runner/kernel_runner.cuh`, parameterised on a `ModeConfig` type `C`. The template runs one warmup iteration followed by N timed iterations of `C::launch_kernel(...)`, accumulates per-iteration kernel time via CUDA events, and queries `C::smem_size()`, `C::smem_breakdown()`, `C::kernel_regs()`, and `C::occupancy_blocks_per_sm()` to record the kernel’s resource fingerprint alongside its timings. The harness never inspects the codec or the kernel body: every codec-specific detail is hidden behind the concept, which is what allows new modes to drop in without modifying the harness.

Table 3.2: The operator contract, realised as a C++20 concept in the standalone benchmark. Every backend in `apps/.../modular/mode/*.cu` satisfies these requirements; the JIT-emitted MDO in Proteus produces the same shape via LLVM IR. The mapping is what lets the standalone benchmark serve as an oracle for the JIT operator.

Operator-level concept (paper)	Standalone-benchmark member
<code>resolve_metadata(tile)</code> hook	Host-built per-tile pointer table + per-tile leader broadcast inside <code>launch_kernel()</code>
<code>decompress_tile(tile)</code> hook	Per-mode kernel template in <code>mode/*.cu</code> , dispatched through <code>decompressor/*.cu</code>
Placement knob (<code>decom_op</code> / <code>decom_reg</code>)	RFOR_GLOBAL vs. RFOR_SHARED modes; FLBP's register-resident decode in <code>fl_bitpack_decompressor.cuh</code>
Self-describing blocks	Per-vector metadata struct in <code>compress/fl_bitpack_compress.hpp</code> + host-built pointer table
Plan-time tile/thread configuration	Template parameters (BT, IPT) with <code>static_assert(BT * IPT == 1024)</code> for FLBP
Format pool	ExecMode enum in <code>runner/run_config.cuh</code> + dispatch in <code>main.cpp</code>

Per-block phase timing. Each kernel times three phases of its own execution using the device intrinsic `globaltimer64()`, with only thread 0 of each block writing the timing arrays to avoid contention:

```

t0 → [decompress all NC columns into regs/smem] → t1
t1 → [apply SSBQ11Functor per element]           → t2
t2 → [CUB BlockReduce + atomicAdd]               → t3

```

The arrays `d_block_decompress_ns`, `d_block_query_ns`, and `d_block_reduce_ns` are copied back after the kernel and fed into an LPT (longest-processing-time) scheduler simulation that distributes the per-block phase triples across $nSMs \times occupancyBlocksPerSM$ virtual workers and reports the critical-path latency. The point of the simulation is to separate algorithmic cost (the sum of phase times) from launch-geometry cost (how those times pack onto the available SMs); the resulting estimate correlates strongly with the wall-clock CUDA-event time in practice, and is the first-order proxy for wall time that the evaluation chapter depends on.

Build and run. `run.sh` builds the `standalone_ssb` target via CMake and runs it with a `MODE` argument (`RFOR_GLOBAL` | `RFOR_SHARED` | `FL_BITPACK` | `FL_DELTA` | `UNCOMPRESSED_ROW` | `ALL`). Output is written to a timestamped CSV under `results/` containing one row per (mode, BT , IPT , iteration) tuple. `sweep.sh` drives a Cartesian-product sweep across (BT, IPT) pairs for FLBP (subject to the $BT \cdot IPT = 1024$ invariant) and across a wider (BT, IPT, T) grid for RFOR.

3.2.2 Codec backends

FLBP: register-resident lane-stream decode. The FLBP backend is the simplest of the codecs because FastLanes’ decode is purely arithmetic on per-thread values. The kernel in `mode/fl_bitpack.cuh` declares a per-thread register array `int vals[IPT]` and loops over *IPT* items per thread, with a single device call per item to `decodeElementFLBPLaneStreamRuntime()` in `decompressor/fl_bitpack_decompressor.cuh`. That decoder computes the FastLanes lane and position from the linear element index at run time, reads exactly one packed `uint32` word from global memory, extracts the bit field, and adds the FOR base broadcast through shared memory by thread 0. Because the entire decode lives in registers, the FLBP backend has no per-block static shared-memory footprint beyond the metadata broadcast slot and the CUB reduction buffer. This is the standalone realisation of the `decom_reg` placement (Section 3.1); FLBP has no analogous FLBP_GLOBAL or FLBP_SHARED variants in the bench because per-thread arithmetic decode has no intermediate buffer that a writeback or shared-memory staging variant could route around. The $BT \cdot IPT == 1024$ invariant is enforced by a `static_assert` in the decompressor header so a misconfigured sweep fails at compile time rather than producing silent garbage.

RFOR with two placement variants. RFOR exists in the benchmark as two distinct modes, RFOR_GLOBAL and RFOR_SHARED, because the placement choice changes more than a buffer pointer in the cooperative decoder. Both modes share the per-tile decoder in `decompressor/rle_binpack_decompressor.cuh`: it stages the bit-packed value stream into shared memory, bit-unpacks it, adds the value-stream FOR base, reuses the staging buffer for the run-length stream, runs a CUB BlockScan to expand runs, and emits one decoded value per element. The two modes differ in what happens next. RFOR_GLOBAL writes those decoded values back to a global-memory tile buffer and the query loop reads them from there: this is the standalone realisation of the `pre_decom` placement (Section 3.1), and serves as the eager-decompression reference point throughout Chapter 4. RFOR_SHARED keeps the decoded values in shared memory and the query reads from there directly: this is the standalone realisation of `decom_op`. The two modes therefore differ in (a) the shared-memory footprint they declare to the harness, (b) the address space the SSB Q1.1 functor loads from, and (c) the achievable occupancy.

Why a separate UNCOMPRESSED_ROW baseline. The UNCOMPRESSED_ROW mode in `mode/uncompressed_row.cuh` runs the same harness, the same query functor, and the same reduction-and-aggregation phases on the four raw `int32` columns. It is the right-most reference column in every chart in Chapter 4: the absolute lower bound on query time once decompression is removed, and the absolute upper bound on GPU memory footprint once compression is removed. Sharing the harness with the compressed modes ensures the only difference is the decompress phase itself.

Correctness. Two checks guard against silent miscoding. First, `tests/test_decompressors.cu` is a GTest binary that round-trips each decoder against random and adversarial input vectors. It is built as a separate target (`modular_decompressor_tests`). Second, `runner/correctness.cuh` computes the SSB Q1.1 revenue from the per-block partial sums after each timed iteration and compares it against the known expected revenue at scale factor 100; any divergence aborts the run and the output CSV is not written. Together these catch both algorithmic regressions in the decoder and silent corruption from wrong (*BT*, *IPT*) configurations.

3.2.3 Metadata management

Two managers, one contract. The two tracks resolve compressed-tile metadata very differently, and the contrast is important for interpreting the cross-comparison in Chapter 4. The standalone benchmark uses a *host-built pointer table*: at compress time, the encoder in `compress/codec_compress.hpp` walks every (`column`, `tile`) pair, computes the metadata struct, allocates the packed payload on the device, and returns an array of (`metadata*`, `packed_data*`) pairs ready for the kernel to index by `tile_idx = blockIdx.x`. The kernel does no parsing at run time. It dereferences the pointer table once per block and broadcasts the metadata to shared memory through thread 0.

The Proteus path uses a *JIT-emitted resolver*. The `CompressedBlockToTuples` (CB2T) operator does not know at plan time where each tile lives in global memory; instead it emits LLVM IR that, at run time, reads the per-block descriptor produced by the storage layer, parses the in-block header, and computes the tile pointer and metadata fields inline inside the same kernel iteration. Thread 0 performs the parse and writes the resolved fields to shared memory; the remaining threads of the CTA wait at a `__syncthreads()` for the broadcast. Both managers ultimately deliver the same metadata to the same decoder, but they pay for it differently: the standalone benchmark pays at compress time, the Proteus path pays at execution time.

Why this asymmetry is acceptable. The asymmetry is deliberate. The host-built pointer table in the standalone benchmark removes a category of cost (runtime metadata resolution) that the Proteus path cannot remove without changing the storage layer. When we report a gap between the two tracks in Chapter 4, this asymmetry is one of the factors that the gap is attributable to - which is exactly what we want from a controlled experiment. The standalone benchmark would no longer be useful as an oracle if it tried to mimic the Proteus resolver, because doing so would import the JIT-emission shape we are trying to measure against.

Table 3.3: The four measurement sources aggregated by the profiling pipeline. Each captures a distinct layer of cost; together they make the cross-comparison between the standalone benchmark and the JIT-emitted Proteus operator falsifiable.

Source	What it captures	Used in
ptxas reports	Compile-time facts per kernel: register count per thread, static shared-memory footprint, spill stores and loads.	Predict-then-verify the occupancy step in Section 4.4.
ncu (Nsight Compute)	Hardware counters at run time: achieved occupancy, eligible warps per scheduler, DRAM read/write throughput, stall reasons.	Mechanism-level findings in Section 4.6.
Wall-clock time	End-to-end execution time per configuration, reported by both the standalone benchmark and Proteus from their own timed loops.	Footprint-time Pareto and configuration sensitivity in Sections 4.2 and 4.4.
Hand-written estimators	Theoretical shared-memory footprint and resident-block count for a given (codec, placement, <i>BT</i> , <i>IPT</i> , <i>T</i>) tuple, computed from each codec’s known decode algorithm.	Sanity-check against ptxas and ncu; explain step-function breakpoints.

3.3 Profiling pipeline

No single tool reports every layer of cost a fused decompression kernel pays: ptxas prints what the compiler decided about registers and shared memory at build time but is silent at run time; ncu captures hardware counters during execution but does not see what ptxas saw; neither reports the wall-clock time a query actually takes; and none of them know whether a configuration *should* have been able to fit a given number of resident blocks. The profiling pipeline used in this report therefore aggregates four complementary sources of measurement (Table 3.3). The aggregation itself is a small set of shell and Python scripts under `results/scripts/` and the two benchmark trees; each script reads files on disk, writes files on disk, and is easily inspected.

Chapter 4

Evaluation

This chapter answers three questions in sequence. *What does fusing decompression into the query pipeline cost end-to-end?* (Section 4.2) *How robust is that cost across the configuration space the operator exposes?* (Sections 4.3 and 4.4) *Where in the JIT pipeline is the next optimisation opportunity?* (Section 4.6). Section 4.5 pauses between the configuration and mechanism findings to validate the standalone benchmark as an oracle. Section 4.7 closes with the synthesis and the limitations.

4.1 Setup

Hardware and software. All experiments run on a dual-socket AMD EPYC 7413 server with 256 GB of host DRAM and a single NVIDIA A40 GPU (sm_86, 48 GB of GDDR6, 10,752 CUDA cores, four warp schedulers per SM). Proteus is built with CUDA 11.7 and Clang/LLVM 14; the standalone benchmark is built with the same toolchain to keep the codec, the data, and the compiler stack identical across tracks. NCU profiles are collected with Nsight Compute 2025.1.

Workload. We use the Star Schema Benchmark (SSB) at scale factor 100. The four queries Q1.1, Q2.1, Q3.1, and Q4.1 collectively scan 9.6 and 14.4 GB of `lineorder` per query and stress different parts of the pipeline: Q1.1 is the simplest with no joins, Q4.1 the join-heaviest. We report end-to-end query time and GPU memory footprint per configuration, along with the relevant ncu and ptxas counters.

Single-kernel-per-pipeline in-GPU mode. Proteus' default mode interleaves data movement and execution at the row-group granularity. For the experiments below, we instead run in *single-kernel-per-pipeline* mode (Section 2.3): all input data is already resident in GPU memory and each compiled pipeline runs as a single kernel launch.

Codecs and configurations. We compare three baselines: uncompressed columns (NONE), Fast-Lanes bitpack (FLBP), and RFOR. Each compressed codec is run under both placement modes (decom_op, decom_reg) and across a sweep over the (BT, IPT, T) triple subject to the $BT \cdot IPT = 1024$ invariant for FLBP. For each (query, codec, placement), the fastest-median configuration is selected by the procedure in `results/scripts/summarize_fastest_codec_configs.py`.

4.2 End-to-end footprint vs. query time

The headline finding. Figure 4.1 plots GPU memory footprint against query execution time across all four SSB queries, with each codec at its fastest configuration. The chart contains the central claim of the work: fused on-chip decompression opens a region of the footprint–time Pareto frontier that no eagerly-decompressed configuration occupies. FLBP and RFOR each contribute a distinct Pareto point per query, and neither dominates the other: FLBP wins on raw query time at moderate projectivity, while RFOR wins on footprint thanks to its richer encoding. The uncompressed baseline is the right-most reference column in each panel: it has the smallest query time but the largest footprint.

Where the savings come from. The footprint reduction is direct: at SF= 100 each lineorder column is 2.4 GB uncompressed; FLBP and RFOR compress these by $\approx 2\text{--}3.5\times$ depending on the column, so the in-GPU working set drops accordingly. The query-time picture is less obvious and explains why eager decompression is not an attractive alternative: writing decoded values back to HBM and re-reading them from downstream operators substantially increases total DRAM traffic, whereas the fused path emits the same final filter–aggregate output without any intermediate

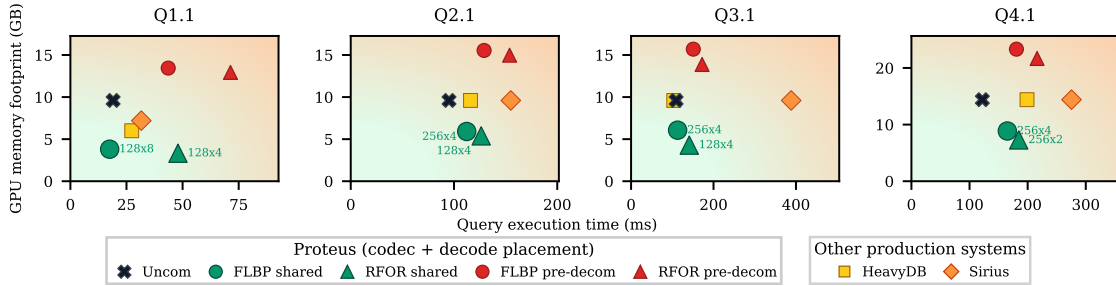


Figure 4.1: GPU memory footprint vs. query execution time across SSB Q1.1–Q4.1 at SF= 100, with each codec at its fastest-median configuration (selected by `summarize_fastest_codec_configs.py`). The background gradient indicates the preferred trade-off direction, from lower time/footprint in green to higher time/footprint in red. FLBP and RFOR each contribute distinct Pareto points that no uncompressed or eagerly-decompressed configuration dominates. The standalone benchmark and Proteus produce matching footprints; query-time differences attributable to the JIT operator are the subject of Section 4.6.

writeback. The DRAM-write reduction is what closes the query-time gap to the uncompressed baseline despite paying for decompression inside the kernel.

Per-codec specialisation. A subtler outcome is that the two codecs occupy different Pareto points not just by tile-decode mechanics but by *which placement they prefer*. FLBP, whose decode is arithmetic on per-thread values, prefers `decom_reg` on scan-heavy queries: the per-thread `ints[IPT]` array sits in registers and downstream operators consume those bindings without round-tripping through shared memory. RFOR prefers `decom_op`: its multi-step decode stages two streams into shared memory anyway, and re-using that scratchpad for the query phase saves both register pressure and an additional smem traversal. This is the operator-level realisation of the design decision in Section 3.1: placement is not a property of the codec in isolation but of $(\text{codec}, \text{query}, \text{projectivity})$ jointly. Section 4.3 quantifies both preferences - and the query-complexity boundary at which the FLBP preference flips with profiler data.

4.3 Placement and launch-configuration sensitivity

Figure 4.1 shows only each codec’s winning configuration; this section examines the two knobs that decide the winner. The profiler evidence is collected in Tables 4.1 and 4.2: *Occupancy a/t* is achieved over theoretical occupancy (%), *Memory BW* and *Compute BW* are memory and SM throughput as a percentage of peak, and *Spill S/L* is the register spill store/load volume in bytes reported by `ptxas` (Section 2.5). Achieved occupancy matches theoretical occupancy within a fraction of a percent in every row, so occupancy is decided entirely at compile time by the resource budgets of Section 2.1 - which is what makes the hand-written shared-memory and register estimators of Section 3.3 predictive.

Table 4.1: Profiling metrics for FLBP at $(BT, IPT) = (128, 8)$ under both placement modes, for SSB Q1.1 and Q4.1. Register-resident decode (`decom_reg`) wins the simple scan but loses the join-heavy query to its own spill traffic.

Metric	Q1.1		Q4.1	
	<code>decom_op</code>	<code>decom_reg</code>	<code>decom_op</code>	<code>decom_reg</code>
Occupancy a/t (%)	41.51 / 41.67	82.96 / 83.33	24.97 / 25.00	83.16 / 83.33
Memory BW (%)	31.58	59.44	21.61	82.24
Compute BW (%)	31.23	53.81	15.21	20.62
Spill S/L (B)	0 / 0	152 / 92	204 / 252	888 / 1024
Exec. time (ms)	27.90	17.30	284.10	342.40

Shared vs. registers: placement tracks query complexity. Table 4.1 shows that the register-resident path pays off only for the simplest query shape. On Q1.1, FLBP improves from 27.90 ms under `decom_op` to 17.30 ms under `decom_reg`: dropping the shared-memory staging area doubles achieved occupancy and lifts both memory and SM throughput, while the spill cost stays negligible (152/92 bytes). On Q4.1 the same switch raises achieved occupancy just as dramatically (24.97% to 83.16%), but the spill volume grows from 204/252 to 888/1024 store/load bytes - decoded values that should live in registers are silently round-tripped through local memory - and execution time *worsens* from 284.10 to 342.40 ms. The same trend holds across the broader SSB suite: once projection and join complexity grow, the downstream operators’ own register demand crowds out the decoded values, and the added register pressure outweighs the benefit of avoiding shared-memory decode state. This is why Figure 4.1 collapses the two placements into a single point per (codec, query), keeping the faster fused variant. The placement of decoded values has been examined by prior work [1] in isolation; the observation specific to a fused compiled engine is that the decision is not codec-local: registers win for decode-dominated pipelines, and shared memory wins once the relational operators contend for the register file.

Higher occupancy is not faster. Table 4.2 makes the complementary point along the launch-configuration axis, holding codec and placement fixed (RFOR, `decom_op`, $T = 512$) and varying only the (BT, IPT) split. The two configurations 128×4 and 256×2 declare nearly identical shared memory per block, yet 256×2 reaches 83.33% theoretical occupancy against 58.33%. If occupancy were the objective, 256×2 should win everywhere. It does not: across Q1.1–Q3.1, 128×4 is faster. Q1.1 already drives memory and compute throughput high enough that additional resident warps have nothing useful to issue, and halving IPT instead buys per-item loop overhead. Q4.1 reverses the verdict: at 128×4 the kernel under-utilises both memory and compute, so the extra warps of 256×2 are productive, and the lower IPT also trims spilling slightly (188/212 to 180/204 bytes); execution time improves from 212.42 to 184.65 ms. Occupancy, in other words, is a budget rather than a target: adding warps helps exactly when the profile shows unused throughput headroom, and costs time when it does not. This is the configuration-space counterpart of the eligible-warps lesson in Section 4.6, and the empirical ground for the methodological take-away in Chapter 5 that

Table 4.2: Profiling metrics for RFOR under `decom_op` at tile size $T = 512$, across two launch configurations ($BT \times IPT$), for SSB Q1.1 and Q4.1. The higher-occupancy configuration is slower on Q1.1–Q3.1 and faster on Q4.1.

Metric	Q1.1		Q4.1	
	128×4	256×2	128×4	256×2
Occupancy a/t (%)	58.21 / 58.33	83.17 / 83.33	41.61 / 41.67	83.18 / 83.33
Memory BW (%)	63.53	65.30	32.48	60.50
Compute BW (%)	63.53	65.30	32.48	45.14
Spill S/L (B)	0 / 0	0 / 0	188 / 212	180 / 204
Exec. time (ms)	47.95	60.04	212.42	184.65

occupancy is a means, not the goal.

4.4 Projectivity sensitivity

The dimension that exposes the trade-offs. Section 4.3 varied placement and launch shape at each query’s native column count; the remaining axis is the query itself. Projectivity - the number of distinct columns the operator is asked to decompress per tile - is the dimension that exposes the on-chip resource trade-offs most sharply, because per-tile shared memory and per-thread register usage scale roughly linearly with it. Figure 4.2 sweeps the projected-column count from one to four and reports query time per (codec, placement, tile size) configuration.

RFOR is tile-size sensitive; FLBP is not. Three observations follow from the sweep. First, RFOR at $T = 1024$ degrades steeply past two columns: the per-tile shared-memory footprint crosses the occupancy step (Section 2.1), drops the resident-block count by one per SM, and the kernel loses roughly an SM’s worth of overlap on each transition. Second, RFOR at $T = 512$ holds its ground: the smaller tile keeps the per-tile shared-memory budget below the step boundary for all four projectivity levels, at the cost of a slightly higher metadata-amortisation overhead. Third, FLBP at $T = 1024$ stays nearly flat across the sweep because the register-resident decode path has no per-column shared-memory term; the only effect of higher projectivity on FLBP is per-thread register pressure, which does not cross the occupancy step in this range.

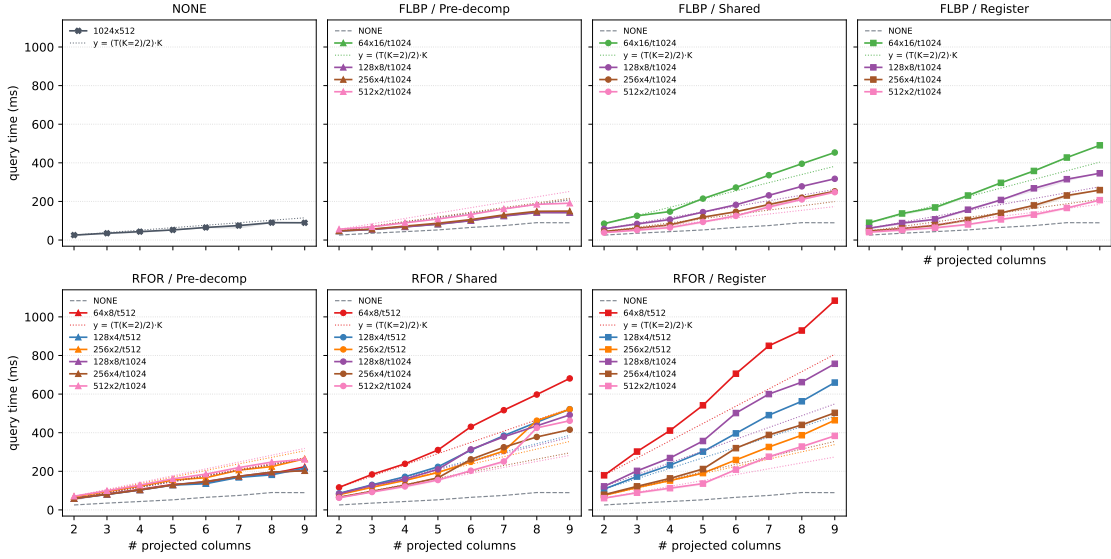


Figure 4.2: Query time vs. projectivity (number of columns the MDO is asked to decompress per tile) across codecs, placements, and tile sizes. RFOR at $T = 1024$ degrades steeply as projectivity grows because each tile stages twice the shared memory; RFOR at $T = 512$ keeps occupancy. FLBP at $T = 1024$ stays close to flat because the per-tile scratchpad cost does not scale with projectivity in the register-resident path.

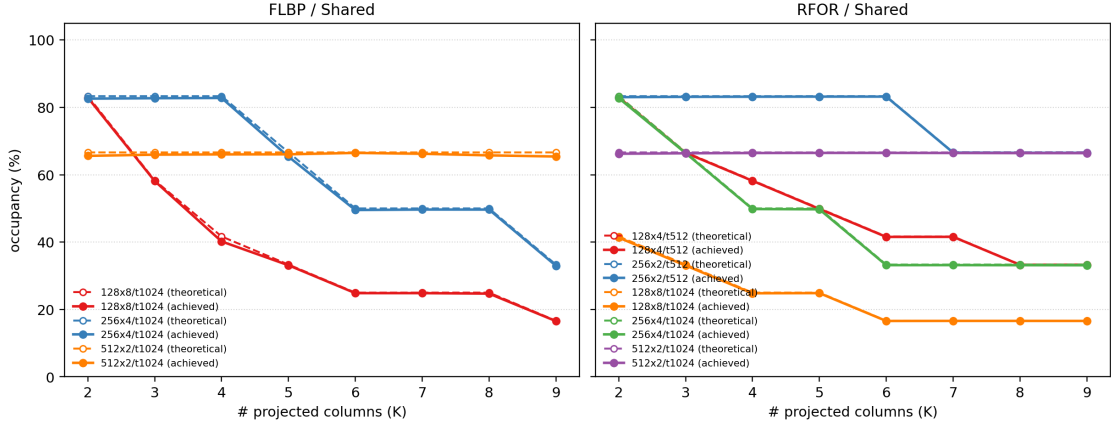


Figure 4.3: Achieved active warps as projectivity grows. The step-function pattern visible on the RFOR $T = 1024$ curve is the resident-block count dropping by one as the per-tile shared-memory footprint crosses a budget boundary.

The occupancy step function in the data. Figure 4.3 shows the underlying mechanism directly: achieved active warps drop by discrete amounts at the configurations where the per-tile shared-memory footprint crosses a budget boundary. Figure 4.4 plots the corresponding shared-memory footprint and lines up the steps exactly with the predictions hand-derived from the codec and decode placement algorithm; the predicted footprints matched the measured values within a single `int32` for every projectivity-level studied.. The agreement within tens of bytes is what makes the occupancy story falsifiable: had the measured smem been higher or lower than predicted, there would have been hidden allocations to track down.

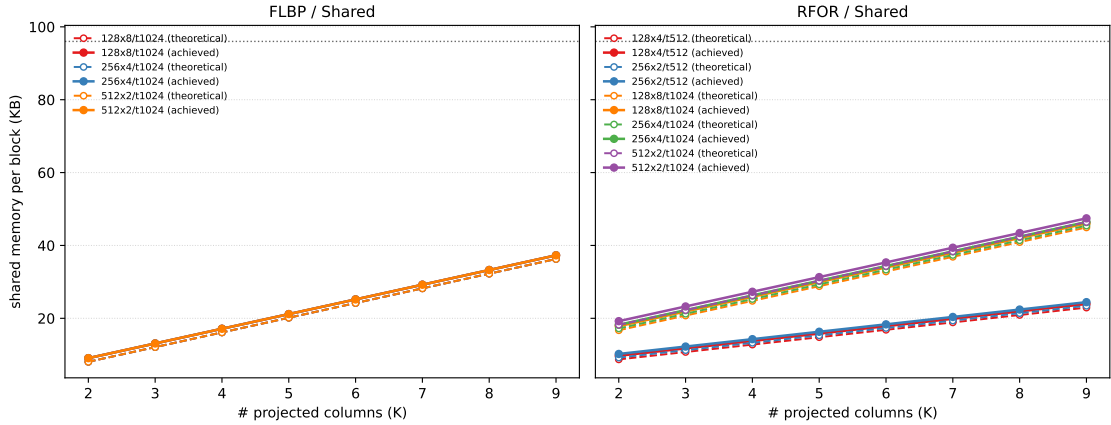


Figure 4.4: Per-block static shared-memory footprint as projectivity grows, by codec/placement/tile size. The breakpoints in Figure 4.3 occur exactly where these curves cross the per-SM scratchpad budget boundary.

Implication for plan-time selection. The result is that the optimal configuration is not a single $(BT, IPT, T, placement)$ point per codec but a small table indexed by projectivity. A query that projects one column is best served by RFOR at $T = 1024$; the same query at four columns prefers RFOR at $T = 512$. FLBP’s flatter response simplifies its plan-time choice but does not eliminate it: at very high projectivity (beyond the four-column reach of this sweep) the register-pressure term catches up.

What does not change. Figure 4.5 confirms that the bandwidth-side ordering of codecs is independent of projectivity: HBM read throughput grows roughly linearly in the number of projected columns, and the relative ordering of NONE, FLBP, RFOR does not invert. The projectivity story is therefore an on-chip-resource story, not a bandwidth story - consistent with the placement-knob design rationale in Section 3.1.

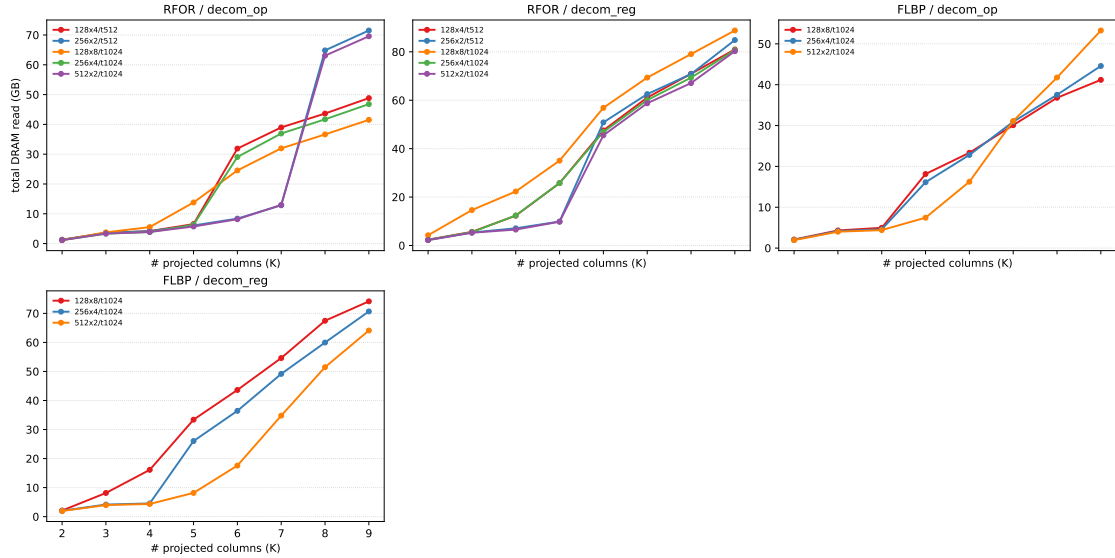


Figure 4.5: DRAM read throughput vs. projectivity. The relative ordering of codecs does not invert with projectivity; the sweep affects on-chip resources, not the HBM-side traffic story.

4.5 Cross-validation: the standalone benchmark as oracle

Why the cross-comparison is necessary. Before reading too much into a JIT-kernel measurement, we should confirm that the standalone benchmark and the JIT operator agree where they are expected to agree. They are configured against the same SF= 100 lineorder data, the same SSB Q1.1 functor, the same FLBP codec, and the same $(BT, IPT) = (128, 8)$ shape. The two kernels do differ in grid geometry - the standalone benchmark processes one FastLanes vector per block (a flat $\approx 586K$ -block grid), Proteus uses a 2D grid of 168×1145 blocks shaped by the row-group structure - so we normalise per block before comparing.

What matches. Table 4.3 reports the ncu metrics from a side-by-side profile collected at matched configuration. The two kernels achieve identical per-thread register counts (48), identical static shared-memory footprints, and effectively identical global-load sector utilisation (79% vs 80%, the small difference coming from a single OID-map load in Proteus that has no analogue in the standalone path). The decoder arithmetic, the FOR-base broadcast, and the bitpack lane access pattern are not distinguishable across the two tracks. This is the sanity-check the standalone benchmark needs to be useful as an oracle: where the design demands they agree, they agree.

What does not match and why. The metrics in the lower half of Table 4.3 are where the two tracks part ways, and they are exactly the metrics that depend on how the kernel was *emitted* rather than what it computes. Both kernels reach the same achieved occupancy (8 active warps per SM), but Proteus’ scheduler sees only 0.56 eligible warps per cycle against the standalone benchmark’s 1.31 - a $2.3\times$ gap in ready-to-issue parallelism at otherwise identical configuration. Section 4.6 explains where the gap comes from.

Table 4.3: Cross-validation of the JIT operator against the standalone benchmark at matched configuration (FLBP, SSB Q1.1, SF= 100, $BT = 128$, $IPT = 8$, $T = 1024$, `decom_reg`, `PROTEUS_NVPTX_MAX_REGS=64`). The two kernels agree on every metric that depends only on the codec and the kernel shape, and disagree on every metric that depends on the JIT IR-emission order inside the pipeline.

Metric	Proteus (JIT)	Standalone (hand)
Per-thread registers	48	48
Static shared memory per block	match	match
Global-load sector utilisation	79%	80%
Achieved active warps per SM (occupancy)	8	8
Eligible warps per scheduler	0.56	1.31
Eligibility per active warp	7.0%	13.3%
Avg. CPI (cycles between issues)	19.9	17.5
L1TEX long-scoreboard stall	11.1 cycles/warp	9.6 cycles/warp

4.6 Mechanism: the eligible-warp gap

The observation. The cross-validation table sets up the puzzle: identical codec, identical kernel shape, identical occupancy, but a $2.3\times$ gap in eligible warps per scheduler. The same gap turns into a measurable runtime difference, with Proteus' Avg. CPI 14% higher than the standalone benchmark's. Something in the JIT path is producing a kernel whose scheduler runs out of work to issue more often than the hand-written equivalent.

The first hypothesis (incorrect). The first explanation we considered was the per-tile meta-data broadcast: in `CompressedBlockToTuples`' `decom_reg` path, only thread 0 of each block parses the tile header, and the remaining threads wait at a `__syncthreads()` until the broadcast completes: the asymmetry was found by reading the IR-emission code in `CompressedBlockToTuplesReg::consume()`. The hypothesis was that the barrier was idling warps and lowering eligibility. The ncu `SCHEDULERSTATS` numbers reject this: the barrier-stall fraction of total CPI is small ($\approx 3\%$), and the long-scoreboard (memory-latency) stall fraction dominates at $\approx 55\%$ in both tracks. The barrier exists, but it is not the bottleneck.

The Potential cause. The actual mechanism is in how the JIT emits the per-tile decode IR. Inside `CompressedBlockToTuples::consume()`, the LLVM IR for decoding the four projected columns is emitted column-by-column: each column's full `resolveTileMetadata()` and `loadTile()` sequence runs to completion before the next column's IR is emitted. ptxas therefore receives a SASS sequence in which each column's loads must complete before the next column's loads can issue, and produces a kernel with four serialised global-load stalls per tile. The hand-written standalone kernel emits a single tight loop over all four columns and a single decode body, which ptxas can pipeline holistically: the SASS schedules all column-0..column-3 loads adjacently and emits a single stall per tile rather than four.

Quantifying the gap. This is what shows up in Table 4.3 as the eligibility-per-warp difference: 7.0% for Proteus and 13.3% for the standalone benchmark. The standalone kernel issues from a larger pool of ready warps at every cycle, because its warps are stalled on fewer, longer-amortised global loads rather than on four short ones in sequence. The long-scoreboard stall is the same memory mechanism in both kernels; only its *count per tile* differs, and that count is set entirely by the IR emission order.

Why this is a JIT-emission opportunity. The crucial property of this finding is its location: the gap is not in the codec, the metadata layout, or the placement choice. It is in the order in which the JIT writes down the IR for the columns of a tile. Switching the JIT to emit all four columns' `loadTile()` bodies into one fused loop - with the metadata broadcasts hoisted out and the per-column decode

bodies interleaved - would close most of the $2.3\times$ eligible-warp gap without touching the codec backends. This is the next unexploited optimisation inside the modular decompression operator and the most concrete recommendation of this project. A full implementation is left to future work; the experimental case for prioritising it is the gap in Table 4.3.

Methodological aside. The investigation followed the predict-then-verify pattern that Section 2.5 introduced. The first hypothesis (the `__syncthreads`) was the visible synchronisation in the IR and the natural guess - and the ncu profile rejected it. The eligible-warp metric is what surfaced the potential cost, and the IR walk-through is what explained where it came from. Without the standalone benchmark as the controlled experiment, the eligible-warp gap would have been ambiguous between "JIT pipeline overhead" and "codec/decoder cost"; with it, the attribution is clean.

4.7 Synthesis and limitations

Synthesis. The three findings stack: fused decompression opens a previously empty region of the footprint-time Pareto (Section 4.2); the plan-time configuration choice is more sensitive than a single fastest-config table can capture and depends on projectivity through the shared-memory step function (Section 4.4); and at matched configuration, the remaining query-time difference between the JIT operator and the hand-written kernel is attributable to JIT IR emission order rather than to anything in the codec or the operator contract (Section 4.6). Together they validate the design decisions in Chapter 3: the two-hook contract is sufficient to ship FLBP and RFOR as plug-and-play backends; the placement and tile/shape knobs are non-redundant; and the standalone benchmark is the right controlled experiment for diagnosing the remaining gap.

Limitations. Three limitations bound the scope of these results. First, all numbers are from a single A40 GPU; the projectivity step heights are `sm_86`-specific and will move under different per-SM register and shared-memory budgets, though the qualitative ordering should be stable. Second, the SSB scale factor and the four queries are a narrow sample of analytical workloads; queries with substantially different join-versus-projection ratios will exercise different parts of the placement trade-off. Third, the configuration sweep is over the discrete set (BT, IPT, T) with the FLBP invariant, not over a continuous parameter space; per-query, per-codec configuration optimisation is an open problem that the plan-time-table observation in Section 4.4 is the entry point for.

Chapter 5

Conclusion

This project (along with the companion paper [2]) showed that lightweight columnar decompression can be a first-class, JIT-fused, plug-and-play component of a compiled tuple-at-a-time GPU query engine. The Modular Decompression Operator (MDO) introduced in Proteus reduces every codec to two backend hooks (`resolve_metadata`, `decompress_tile`) and exposes on-chip placement and tile/thread shape as orthogonal configuration knobs. The standalone benchmark (`apps/standalones/adm/adm-encode-bench/modular/`) implements the same contract by hand, which is what makes the JIT operator’s hardware-level behaviour diagnosable as a controlled experiment rather than as a black box.

Two findings. The empirical study produced two results that depend on each other. **(i)** Fused on-chip decompression opens a previously empty region of the GPU memory-footprint–query-time Pareto frontier across SSB Q1.1–Q4.1 (Section 4.2); reaching the best point in that region is not free, however, because no single configuration wins across queries - RFOR’s heavier decoder hits on-chip resource limits sooner than FLBP’s lighter one as the number of projected columns grows, so the optimal (BT, IPT, T) shifts with the workload (Section 4.4). **(ii)** At matched configuration and codec, the JIT-fused kernel achieves only 0.56 eligible warps per scheduler against the standalone benchmark’s 1.31 (Section 4.6). The gap is not in the algorithm or the codec; it is in how the JIT emits per-column decode IR sequentially, preventing ptxas from pipelining loads across columns. A single change to that emission shape is the next unexploited optimisation inside the operator.

Methodological take-aways. The investigation also produced four methodological lessons that generalise beyond this work:

- *It is always a tradeoff, choose wisely.* Placement, tile size, (BT, IPT) , and codec each trade one on-chip resource against another, and no configuration dominates across queries. The

plan-time question worth asking is therefore not which configuration is fastest, but which resource the workload can afford to spend; the answer changes with the workload, and the engineering work is to choose deliberately rather than to search for a free lunch.

- *Predict before profiling:* If the predicted shared-memory or register count cannot be sketched on paper, the measurement is not yet interpretable. The discrepancies are where the learning happens.
- *Build the controlled experiment:* The standalone benchmark paid for itself the first time it disagreed with Proteus; without it, the eligible-warp gap would have been ambiguous between codec, contract, and JIT cost.
- *Trust `ptxas` as much as `ncu`:* Register and spill reports caught configurations that `ncu`'s averaged metrics smoothed over.
- *The visible cost is rarely the whole cost:* Barriers, occupancy, tile size, and the `__syncthreads` after thread-0 broadcast each looked like the bottleneck at one point but were not borne out by the profile data. The strongest remaining candidate is warp eligibility, plausibly shaped by IR emission order; confirming the causal chain rigorously is left to future work.

Future work. Three directions follow directly from the findings. **Cross-column load interleaving in the JIT.** Changing `CompressedBlockToTuples::consume()` to emit all projected columns' `loadTile()` bodies into one fused loop - with the metadata broadcasts hoisted out and the per-column decode bodies interleaved - is the most concrete prescription this project can offer. The experimental case for prioritising it is the $2.3\times$ eligible-warp gap in Table 4.3. **Per-column or per-block codec selection.** The MDO contract admits per-column codec choices without operator modification [2]; integrating a plan-time codec selector (possibly with a learned cost model [10]) would extend the Pareto frontier further on workloads with heterogeneous value distributions. **More codecs on the same contract.** The two-hook contract was designed to make new codecs cheap to add. Patching in ALP, Patas, or a delta-of-delta variant should require only one new `ModeConfig` in the standalone benchmark plus one new backend class in Proteus - a worked example would validate the "plug-and-play" claim under broader codec diversity.

Closing. The broader point is that GPU memory is becoming the binding resource for analytical workloads, and that the response cannot be a one-off compressed-execution kernel built for a specific codec or a specific query model. Treating decompression as a first-class compiler abstraction - shaped by, and measured against, a hand-written controlled experiment - is what makes compressed execution composable with the rest of a compiled GPU engine, and what makes its remaining costs diagnosable rather than mysterious.

Bibliography

- [1] Azim Afroozeh, Lotte Feliuss, and Peter Boncz. “Accelerating GPU Data Processing using FastLanes Compression”. In: *Proceedings of the 20th International Workshop on Data Management on New Hardware*. DaMoN ’24. New York, NY, USA: Association for Computing Machinery, 2024. ISBN: 9798400706677. DOI: 10.1145/3662010.3663450. URL: <https://doi.org/10.1145/3662010.3663450>.
- [2] Antonio Boffa, Ananya Gupta, Hamish Nicholson, and Anastasia Ailamaki. “Tile-Based Decompression in Compiled GPU Engines”. Companion paper. Manuscript in preparation. 2026.
- [3] Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. “Het-Exchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines”. In: *Proc. VLDB Endow.* 12.5 (2019), pp. 544–556. ISSN: 2150-8097. DOI: 10.14778/3303753.3303760. URL: <https://doi.org/10.14778/3303753.3303760>.
- [4] *CUDA C++ Programming Guide*. 2026. URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [5] Henning Funke, Sebastian Breß, Stefan Noll, Volker Markl, and Jens Teubner. “Pipelined Query Processing in Coprocessor Environments”. In: *Proceedings of the 2018 International Conference on Management of Data*. SIGMOD ’18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 1603–1618. ISBN: 9781450347037. DOI: 10.1145/3183713.3183734. URL: <https://doi.org/10.1145/3183713.3183734>.
- [6] *HeavyDB*. 2024. URL: <https://github.com/heavyai/heavydb>.
- [7] Zezhou Huang, Krystian Sakowski, Hans Lehnert, Wei Cui, Carlo Curino, Matteo Interlandi, Marius Dumitru, and Rathijit Sen. *GPU Acceleration of SQL Analytics on Compressed Data*. 2025. arXiv: 2506.10092 [cs.DB]. URL: <https://arxiv.org/abs/2506.10092>.
- [8] *Multi-Instance GPU (MIG)*. 2026. URL: <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>.
- [9] *Multi-Process Service (MPS)*. 2026. URL: <https://docs.nvidia.com/deploy/mps/index.html>.

- [10] Thomas Neumann. “Efficiently compiling efficient query plans for modern hardware”. In: *Proc. VLDB Endow.* 4.9 (2011), pp. 539–550. ISSN: 2150-8097. DOI: 10.14778/2002938.2002940. URL: <https://doi.org/10.14778/2002938.2002940>.
- [11] Hamish Nicholson, Konstantinos Chasialis, Antonio Boffa, and Anastasia Ailamaki. “The Effectiveness of Compression for GPU-Accelerated Queries on Out-of-Memory Datasets”. In: *Proceedings of the 21st International Workshop on Data Management on New Hardware*. DaMoN ’25. New York, NY, USA: Association for Computing Machinery, 2025. ISBN: 9798400719400. DOI: 10.1145/3736227.3736240. URL: <https://doi.org/10.1145/3736227.3736240>.
- [12] *nvCOMP*. 2024. URL: <https://docs.nvidia.com/cuda/nvcomp/>.
- [13] *NVIDIA Ampere GA102 GPU Architecture*. 2021. URL: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>.
- [14] Johns Paul, Shengliang Lu, and Bingsheng He. “Database Systems on GPUs”. In: *Found. Trends Databases* 11.1 (2021). ISSN: 1931-7883. DOI: 10.1561/19000000076. URL: <https://doi.org/10.1561/19000000076>.
- [15] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. “A Study of the Fundamental Performance Characteristics of GPUs and CPUs for Database Analytics”. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD ’20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 1617–1632. ISBN: 9781450367356. DOI: 10.1145/3318464.3380595. URL: <https://doi.org/10.1145/3318464.3380595>.
- [16] Anil Shanbhag, Bobbi W. Yogatama, Xiangyao Yu, and Samuel Madden. “Tile-Based Lightweight Integer Compression in GPU”. In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD ’22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 1390–1403. ISBN: 9781450392495. DOI: 10.1145/3514221.3526132. URL: <https://doi.org/10.1145/3514221.3526132>.
- [17] Bobbi Yogatama, Yifei Yang, Kevin Kristensen, Devesh Sarda, Abigale Kim, Adrian Cockcroft, Yu Teng, Joshua Patterson, Gregory Kimball, Wes McKinney, Weiwei Gong, and Xiangyao Yu. “Rethinking Analytical Processing in the GPU Era”. In: *16th Conference on Innovative Data Systems Research, CIDR 2026, Chaminade, CA, USA, January 18-21, 2026*. www.cidrdb.org, 2026. URL: <https://vldb.org/cidrdb/2026/rethinking-analytical-processing-in-the-gpu-era.html>.